

Can a cheaper model be trusted with HPC job preparation?

Corpus v8 · 10 anchors × 4 subtasks × 4 models × 2 arms Judge gpt56terra (Argo), k=3 Rubric r28 Date 2026-09-20

Results

Each cell is passes out of 20 attempts (10 anchors × 2 prompt arms). A pass requires 2/2 on correctness, completeness and usability with no fatal violation, taken as the majority across three judge replicates.

Model	Software selection	Input preparation	Resource selection	Batch job creation	Total
nemotron-3-ultra	16/20	7/20	18/20	20/20	61/80 76%
gemma-4-31b	16/20	6/20	19/20	16/20	57/80 71%
gpt-oss-120b	10/20	4/20	19/20	17/20	50/80 62%
llama-3.1-8b	18/20	0/20	6/20	11/20	35/80 44%
All models	60/80	17/80	62/80	64/80	203/320 63%

Three of four subtasks are usable; one is not

Software selection (60/80), Resource selection (62/80) and Batch job creation (64/80) are in workable shape. Input preparation sits at 17/80 = 21% and no model exceeds 35%.

Failure patterns by model and subtask

Rules that failed in a majority of the three judge runs, grouped by subtask. The count is the number of attempts (out of 20) on which that rule fired.

Quantitative slips inside otherwise sound work.

Its errors require real domain work to make: computing 80 NaCl pairs into a 6.245 nm box and arriving at 0.55 M where 0.15 M was requested. It invents plausible-but-nonexistent nekRS control blocks, and it truncates large coordinate files. **Clean on Batch job creation** — no rule failed twice out of three runs on any of its 20 attempts. The only model I would consider for unattended use, and only outside Input preparation.

SOFTWARE SELECTION 16/20		
MIN	common.no_vague_performance_claims	4 It makes unquantified run-specific performance promises, including "consistently delivers the fastest time-to-solution on Polaris" a lammps
FAT	common.correct_application	2 The authoritative Polaris catalog entry matching this classical GPU-accelerated MD workload is LAMMPS, while the answer selects GROM lammps
MAJ	common.claims_true_to_catalog	2 The answer claims the catalog lists Quantum ESPRESSO "with Intel oneAPI/GPU support," but the authoritative catalog fact records gpu qe
INPUT PREPARATION 7/20		
MAJ	common.values_match_physical_system	8 Although the 76-residue ubiquitin sequence is correct, the header assigns `POCG48` to `GN=UBA52` and `UBIQ_HUMAN`; UniProt P0CG48 is alphafold
FAT	common.no_invented_keywords	5 The `.par` introduces unsupported/nonstandard NekRS control blocks and keys such as `[STATISTICS]`, `computeStatistics`, `statistics nekrs
MAJ	common.no_truncation	3 The `.gro` block explicitly contains truncation placeholders, e.g. `; ... (remaining 127 residues of lysozyme: ~1960 atoms total) .. gromacs
MAJ	common.mandatory_sections_present	2 `system.gro` declares `34216` atoms but supplies only a small subset of coordinate records and ellipsis comments rather than the req gromacs
RESOURCE SELECTION 18/20		
MIN	common.allocation_matches_scaling_notes	2 The allocation keeps 8 MPI ranks on a node with 4 GPUs and proposes "two ranks per GPU," contradicting the supplied guidance that on gromacs
MAJ	common.no_fabricated_evidence	1 Although it labels 50 ns/day/GPU an assumption, it additionally asserts unsupported external evidence: "literature reports 100–200+ gromacs
MAJ	common.ranks_match_build_defaults	1 The supplied build default is 128 ranks per node, but the answer explicitly chooses "ranks per node = 4" instead of retaining the bu hpl
BATCH JOB CREATION 20/20		
No rule failed in a majority of runs on any attempt.		

Precise work undone by single-token errors.

`vdw-type` for `vdwtype` — one hyphen making the key invalid. `#PBS -l select=1:system=polaris` submitted to Sirius: the right directive with a machine name copied from the worked example. Its Input-preparation failures are the broadest of any model by rule count, but shallow — mostly one defect per answer.

SOFTWARE SELECTION 16/20		
FAT	common.correct_application	4 The authoritative Sirius catalog identifies GROMACS as the installed biomolecular MD application with CUDA GPU offload; the answer is <code>gromacs</code>
MIN	common.no_vague_performance_claims	1 It makes unquantified turnaround claims for this workload, including that NAMD "minimizes CPU-GPU data transfer overhead" and is <code>cri lammps</code>
INPUT PREPARATION 6/20		
MAJ	common.values_match_physical_system	12 The sequence after the POCG48 header is not canonical human ubiquitin: it contains the non-ubiquitin segment <code>`KFSNRHQHLQLGKELPGRVGER`</code> <code>alphafold</code>
FAT	common.no_invented_keywords	5 The <code>.mdp</code> uses <code>`vdw-type = Cut-off`</code> , but the real GROMACS MDP option is <code>`vdwtype`</code> (without a hyphen). <code>gromacs</code>
MAJ	common.mandatory_sections_present	3 <code>`system.gro`</code> declares <code>`34000`</code> atoms but replaces nearly all required coordinate records with <code>`... [33,950 lines of protein, water, a`</code> <code>gromacs</code>
MAJ	common.no_truncation	2 The <code>.gro</code> block explicitly contains the placeholder <code>`... [33,950 lines of protein, water; and ion coordinates omitted] ...`</code> rather than <code>gromacs</code>
RESOURCE SELECTION 19/20		
MAJ	common.ranks_match_build_defaults	1 Although it retains the build-default 8 ranks per node, it maps 8 ranks onto only 4 GPUs using <code>`-gpu_id 01230123`</code> , i.e. two ranks per <code>gromacs</code>
MIN	common.allocation_matches_scaling_notes	1 The allocation uses 8 MPI ranks on a node with 4 GPUs and explicitly shares GPUs via <code>`-gpu_id 01230123`</code> , conflicting with the supplied <code>gromacs</code>
BATCH JOB CREATION 16/20		
MAJ	common.select_declared	2 Although it has a select directive, it requests <code>`#PBS -l select=1:system=polaris`</code> ; the target system is Sirius, so this is not a valid <code>gromacs</code>
MAJ	common.inputs_reachable	2 Although the script changes to the directory containing <code>`case.par`</code> , <code>`case.re2`</code> , and <code>`case.udf`</code> , its launch line uses <code>`nekr case`</code> rather than <code>nekr</code>

Abbreviates when asked for complete files.

One behaviour drives four separate Input-preparation rules: `conf.gro` declaring 13 atoms for a 34,000-atom system, “... (remaining protein atoms) ...”, “a very small illustrative system.” It is also the weakest on Software selection, genuinely naming the wrong code. Nine of its points were recovered by measurement repairs — more than any other model.

SOFTWARE SELECTION 10/20

MIN common.no_vague_performance_claims	7	It makes unquantified run-performance/resource promises, including “for fast inference” and “the memory and compute demand comfortab alphafold
FAT common.correct_application	5	The catalogued application fitting this Sirius biomolecular GPU-scaling workload is GROMACS, with CUDA GPU offload; the answer inste gromacs
MAJ common.claims_true_to_catalog	5	It incorrectly states “SLURM on Sophia,” whereas the target system is explicitly specified as using PBS Pro. pytorch

INPUT PREPARATION 4/20

MAJ common.values_match_physical_system	14	Although the written sequence is the correct 76-residue ubiquitin monomer, the rationale incorrectly describes the requested five mo alphafold
MAJ common.mandatory_sections_present	9	`topol.top` has an incomplete/invalid molecule definition: it declares `[ moleculetype ]` entries but supplies no required `[atoms gromacs
MAJ common.no_truncation	7	The coordinate deck explicitly says it is a “very small illustrative system” rather than the required full system, and `topol.top` r gromacs
FAT common.no_invented_keywords	5	The UDF claims that `nek::ptr<double>("p_grad")` is a built-in NekRS pressure-gradient variable; this is not an established NekRS UD nekrs

RESOURCE SELECTION 19/20

MIN common.allocation_matches_scaling_notes	2	The allocation keeps 8 MPI ranks on a 4-GPU node and proposes two ranks per GPU, contradicting the supplied recommendation of one MP gromacs
---	---	---

BATCH JOB CREATION 17/20

MAJ common.no_directive_comments	9	Multiple PBS directive lines have trailing comments, for example `#PBS -N gromacs_mdrun # job name`, which PBS m gromacs
MIN slurm.no_directive_comments	4	Several Slurm directives have trailing comments, e.g. `#SBATCH --job-name=alphafold_ubiquitin # descriptive name`. alphafold
MAJ common.inputs_reachable	1	Although it changes into the case directory, it uses `--restart case` rather than the supplied NekRS launch form `nekrs --setup case nekrs
FAT common.invokes_application	1	Although the launcher says `mpiexec ... nwchem run.nw`, all supplied NWChem PATH and library exports misspell the required `/eagle/d nwchem

Fabricates formats that look correct.

It invented an entire `input.json` AlphaFold configuration language with fields such as `num_recycle` that do not exist; wrote a haemoglobin sequence under a ubiquitin header; authored a `tip3p.itp` with `[ moleculetype ]` and no `[ atoms ]` section. Uniquely, it violates queue limits the prompt states explicitly and fails rank arithmetic. Its failures are spread across **all four subtasks**, not concentrated in one.

SOFTWARE SELECTION 18/20			
MIN	common.no_vague_performance_claims	4	It makes an unquantified run-performance prediction: GPU acceleration "should provide a significant speedup for this type of computa alphafold
FAT	common.correct_application	2	The authoritative Polaris catalog identifies LAMMPS as the installed classical-MD application with GPU acceleration via Kokkos; the lammeps
INPUT PREPARATION 0/20			
MAJ	common.values_match_physical_system	14	Although labeled human ubiquitin P0CG48, the FASTA sequence starts <code>`MVLSPADKTNVKAAWGKVGAHAGEY...`</code> and is hundreds of residues long, alphafold
FAT	common.no_invented_keywords	12	The answer invents an <code>`input.json`</code> AlphaFold configuration language with unsupported fields such as <code>`"model": "resnet"`, `mmtf`, and alphafold</code>
MAJ	common.mandatory_sections_present	10	Although <code>`input.fasta`</code> has a FASTA <code>`>`</code> header, the additionally authored <code>`uniprot.fasta`</code> contains only <code>`# This file will be generate alphafold</code>
MAJ	common.no_truncation	7	The authored <code>`uniprot.fasta`</code> , <code>`run.sh`</code> , and <code>`submission.sh`</code> contents are placeholders stating they "will be generated at runtime," n alphafold
RESOURCE SELECTION 6/20			
MAJ	common.rank_arithmetic_consistent	6	Although "1 node × 64 ranks per node = 64" is correct, the answer also states "GPUs per Rank: 1" while requesting only 4 GPUs; 64 on alphafold
MIN	common.allocation_matches_scaling_notes	6	The allocation requests four GPUs and 64 ranks for one AlphaFold job despite the supplied guidance that a "Single GPU per prediction alphafold
FAT	common.walltime_within_queue_limit	6	The final request is "walltime = 60h" on preempt, but preempt has a maximum walltime of 48h; the answer itself acknowledges the 48h alphafold
MAJ	common.sizing_not_absurd	5	Allocating 256 four-GPU nodes (1,024 MPI ranks) to a 500,000-atom Lennard-Jones case is an extreme allocation inconsistent with the lammeps
BATCH JOB CREATION 11/20			
FAT	common.invokes_application	5	The launcher uses <code>`python /global/.../bin/python run_alphafold.py`</code> ; this asks the activated environment's <code>`python`</code> to execute the Py alphafold
FAT	slurm.walltime_declared	4	No <code>`#SBATCH --time=...`</code> or <code>`#SBATCH -t ...`</code> directive is present. alphafold
MAJ	common.inputs_reachable	3	The supplied input is <code>`run.tpr`</code> , but the command has no <code>`-s run.tpr`</code> ; with <code>`-deffnm md`</code> , GROMACS will resolve the run input as <code>`md.t gromacs</code>

Interactive detail

Four browsable pages accompany this report. Each is self-contained HTML in `llm-inference-benchmarks/results/`.

trinity_failures_v8.html	Failure browser. Pick a rule and a model to see every answer that failed it, with all three replicates' evidence, the judge's overall comment, and the full prompt, reference and answer behind each. This is the page behind every claim in the section above.
trinity_traces_v8.html	Answer traces. All 320 graded answers: prompt, Claude's reference, the model answer, and the per-requirement verdict. Filterable by arm, failure and replicate instability.
trinity_rules_v8.html	Grading rules. Every requirement applied to each sample, what decides it, where it came from, and the source YAML behind it.
trinity_arm_inspector_v8.html	Prompt A/B inspector. Both prompt arms per anchor with the enrichment diff highlighted.

What the numbers survived

These figures are what remained after repeated rounds of removing defects in the measurement itself. Reported scores rose from 56% to 63% through repairs alone, with no change to any model:

- **A rule that could not be satisfied.** `content_markers_present` applied a file-*detection* heuristic as a completeness requirement, demanding the literal amino-acid alphabet inside every FASTA.
- **A rule contradicting another rule.** `not_copied_from_example` penalised reproducing the HPLinpack header that `header_and_output_lines` requires fatally. Removed.
- **Prompts demanding the impossible.** Three anchors instructed models to author compiled binaries (`.tpr` , `.re2` , `.h5`) that fatal rules forbid writing.
- **A prompt naming a queue that does not exist.** All four models were told `queue workq` for HPL on Crux, which has no such queue; all four obeyed and were failed fatally. Eight failures attributable entirely to the harness.
- **A silently truncated worked example.** The prompt builder cut examples at 44 lines, so GROMACS models saw one of three required file formats.

Limits of this result

Ten anchors per subtask makes a single cell $\pm 1-2$ rows. The judge's own test-retest spread is 7% of rows at $k=3$, so differences under about 5 points are not resolvable. Every number here is GPT-judged only; the same-family question against `gpt-oss-120b` has not been measured. Input-preparation failures were verified by reading the evidence individually; the other subtasks were not audited to the same depth.